

Domestic Service Robot: Basic Coffee Preparation

Assignment D2-V1 Classical PDDL and PDDL+

Richard Albert King Mechoda

MSc Robotics Engineering, University of Genova
Artificial Intelligence for Robotics II

Problem and solution

The problem

- A fixed manipulator must prepare a cup of coffee in a kitchen
- Required sequence:
 - obtain the empty cup
 - fill it with water
 - add coffee powder
 - brew with the coffee machine
- Constraints:
 - static kitchen, fixed object positions
 - one object handled at a time
- Q1: classical PDDL, two instances with valid plans
- Q2: PDDL+, cooling process, cold event, timing

The solution

- Cup contents modelled as a chain of states; recipe order comes from action preconditions
- Tap, dispenser and machine are location capabilities; only the cup is grasped
- grasp and place are explicit actions on a one-slot gripper
- PDDL+: temperature fluent, autonomous cooling process, event that makes cold coffee unusable
- Checked with a real planner:
 - Q1 with pyperplan, plans of length 3 and 9
 - Q2 with ENHSP, fast delivery solved, slow delivery unsolvable

Q1: classical PDDL model

Ordering comes from preconditions, not from a script

Object states form one chain

```
(empty ?c) -> (has-water ?c) -> (has-powder ?c) -> (coffee-ready ?c)
```

- Stations are capabilities of locations: is-water-source, is-powder-source, is-machine. Only the cup is grasped.
- grasp and place are explicit; every station operation needs the cup placed there with the gripper free

How preconditions force the order

```
fill-water    needs (empty ?c)          only an empty cup can be filled
add-powder    needs (has-water ?c)       powder strictly after water
brew-coffee   needs (has-water ?c) and (has-powder ?c)
```

Q1: instances and plans

Same domain, only the initial state changes the difficulty

Simple: one counter does everything

```
(fill-water cup1 counter)
(add-powder cup1 counter)
(brew-coffee cup1 counter)
```

pyperplan: plan length 3

Complex: separate stations, cup on a shelf

```
(grasp cup1 shelf)
(place cup1 water-source)
(fill-water cup1 water-source)
(grasp cup1 water-source)
(place cup1 powder-dispenser)
(add-powder cup1 powder-dispenser)
(grasp cup1 powder-dispenser)
(place cup1 coffee-machine)
(brew-coffee cup1 coffee-machine)
```

pyperplan: plan length 9

- Planning difficulty is in the structure of the instance, not in the actions

Q2: PDDL+ model

The world changes even when the robot does nothing

Process: continuous cooling

```
(:process cooling
  :precondition (coffee-ready ?c)
  :effect (decrease
    (temperature ?c) (* #t 1)))
```

Event: fires by itself at the threshold

```
(:event coffee-got-cold
  :precondition (and (coffee-ready ?c)
    (<= (temperature ?c)
      (drinkable-temp)))
  :effect (and (not (coffee-ready ?c))
    (spoiled ?c)))
```

- Brewing and delivery take time. ENHSP has no durative actions, so each becomes start action, clock process and completion event.
- Delivery completes only if the coffee is still hot when the hand-over ends

```
start-brew -> brewing-process (clock) -> brew-done at t = 5: temperature := 90
```

Q2: timing decides plan validity

Two instances, identical except one number: delivery-time

delivery-time = 10 -> Problem Solved

```
0.0  fill-water, add-powder,  
      start-brew  
5.0  [brew-done: 90]  
      grasp, start-delivery  
15.0 [delivery-complete ~80]  
      goal (served cup1) reached
```

delivery-time = 70 -> Problem unsolvable

```
hot window: 90 -> 40 at 1 per unit  
             = 50 units < 70 needed  
coffee-got-cold fires in transit,  
delivery-complete can never trigger  
  
ENHSP: "Problem unsolvable"
```

- The planner even found a loophole in the first model: it put the cup back in the machine mid-delivery and re-brewed the spoiled coffee
 - fix: place forbidden while delivering, spoiling made irreversible. A model must encode all its invariants.

What I learned

- State predicates are causal milestones, not physics. Anything left implicit is something the planner can exploit.
- Correct ordering needs no script: preconditions alone give the 3-step and 9-step plans.
- PDDL+ changes the game: processes run without the planner, events fire at thresholds, and waiting is no longer free.
- One duration value flips an instance from solvable to provably unsolvable. That is timing affecting plan validity.
- Practical notes: ENHSP has no durative actions, reserves the name at, and discretises the dynamics with a 1-second step.

Repository: github.com/albertrichard080/D2-V1-Domestic-Service-Robot-Basic-Coffee-Preparation